

```
#!/usr/bin/env python3
```

```
"""
```

```
nanoclaw_router.py
```

```
-----
```

NovusÆxenti / NovÆcopia Autonomous Orchestration & Auditing Stack
Component: NanoClaw Routing Engine & Dynamic Skill/Tool Hot-Loader

Responsibilities:

1. Evaluates incoming user prompts and extracts intent entities and task domains.
2. Implements the Hybrid Architectural Spine:
 - "One Skill to Rule Them All": Invariant governance policy injected as fixed base prefix.
 - "mem-search": Dynamic, on-demand lookup against #d.u.m.b.a.s.s. & local manifest.jsonl files.
3. Dynamically hot-loads relevant tools (from 04_skills_runtime/extracted_tools) and skills (from prompt_skills).
4. Emits compressed, token-optimized context blocks for Claude Code CLI or Prime Agent runtime.

```
"""
```

```
import os
```

```
import sys
```

```
import json
```

```
from pathlib import Path
```

```
WORKSPACE_ROOT = Path(os.path.expanduser("~/novae-xorpus"))
```

```
SKILLS_DIR = WORKSPACE_ROOT / "04_skills_runtime" / "prompt_skills"
```

```
TOOLS_DIR = WORKSPACE_ROOT / "04_skills_runtime" / "extracted_tools"
```

```
MANIFEST_FILE = WORKSPACE_ROOT / "04_skills_runtime" / "manifest.jsonl"
```

```
META_SKILL_POLICY = """
```

```
<system_governance_invariants>
```

1. ZERO TOUCH TO ORIGINALS: Never overwrite, rename, or move original canonical files.
2. SENSORY IMMUTABILITY: 01_raw_sources is strictly read-only.
3. EXPLICIT DUAL EXTRACTION: Always prioritize existing tools in tools/ and prompt_skills/.
4. TELEMETRY COUPLING: All tool execution traces must be mirrored to episodic telemetry.

```
</system_governance_invariants>
```

```
"""
```

```
class NanoClawRouter:
```

```
    def __init__(self, workspace_root: Path = WORKSPACE_ROOT):
```

```
        self.workspace_root = workspace_root
```

```
        self.skill_registry = {}
```

```
        self.tool_registry = {}
```

```
self._load_local_manifests()
```

```
def _load_local_manifests(self):
```

```
    self.skill_registry = {
        "grill-me": {
            "name": "grill-me",
            "description": "Aggressive question-based specification refining before building.",
            "keywords": ["grill", "clarify", "requirements", "interview", "review spec", "questions"]
        },
        "spec": {
            "name": "spec",
            "description": "Architectural specification drafting and design contracts.",
            "keywords": ["spec", "architecture", "blueprint", "contract", "design"]
        },
        "ticket": {
            "name": "ticket",
            "description": "Atomic issue and milestone task ticket creation.",
            "keywords": ["ticket", "task", "milestone", "breakdown", "todo"]
        },
        "tdd": {
            "name": "tdd",
            "description": "Test-driven development implementation and verification harnesses.",
            "keywords": ["tdd", "test", "unit test", "verify", "harness", "assert"]
        },
        "code-review": {
            "name": "code-review",
            "description": "Strict AST-based code review and dependency risk analysis.",
            "keywords": ["code-review", "ast", "review", "audit code", "check code"]
        }
    }
```

```
self.tool_registry = {
```

```
    "npu_manager": {
        "script": "tools/npu_manager.py",
        "keywords": ["npu", "model switch", "load model", "unload model", "qwen", "geniex"]
    },
    "compile_manifest": {
        "script": "tools/compile_manifest.py",
        "keywords": ["manifest", "hash", "sha256", "rebuild index", "catalog"]
    },
    "system_housekeeper": {
        "script": "tools/system_housekeeper.sh",
        "keywords": ["housekeeper", "hygiene", "symlink", "vault boundaries", "sweep"]
    }
```

```

    },
    "graph_builder": {
        "script": "skills-and-capabilities/code-review-graph/graph_builder.py",
        "keywords": ["ast", "dependency graph", "blast radius", "fan-in", "fan-out"]
    }
}

```

```

def route_prompt(self, prompt: str) -> dict:

```

```

    prompt_lower = prompt.lower()
    matched_skills = []
    matched_tools = []

```

```

    for skill_id, meta in self.skill_registry.items():
        if any(kw in prompt_lower for kw in meta["keywords"]):
            matched_skills.append(skill_id)

```

```

    for tool_id, meta in self.tool_registry.items():
        if any(kw in prompt_lower for kw in meta["keywords"]):
            matched_tools.append(meta["script"])

```

```

    return {
        "query": prompt,
        "meta_policy": META_SKILL_POLICY.strip(),
        "hot_loaded_skills": matched_skills,
        "hot_loaded_tools": matched_tools,
        "token_estimate": len(prompt.split()) + 150
    }

```

```

def render_system_prompt_prefix(self, route_result: dict) -> str:

```

```

    skills_str = ", ".join(route_result["hot_loaded_skills"]) if route_result["hot_loaded_skills"] else
    "None (General Mode)"
    tools_str = "\n".join([" - " + t for t in route_result["hot_loaded_tools"]]) if
    route_result["hot_loaded_tools"] else " - Standard workspace CLI"

```

```

    lines = [
        "### NANOCLAW DYNAMIC HARNESS INJECTION",
        route_result["meta_policy"],
        "",
        f"[ACTIVE SKILLS ACTIVATED]: {skills_str}",
        "[ACTIVE TOOLS HOT-LOADED]:",
        tools_str
    ]
    return "\n".join(lines)

```

```
if __name__ == "__main__":  
    router = NanoClawRouter()  
    print("[*] NanoClaw Router active.")
```